

RTL Design Sherpa

STREAM on the Genesys 2 The FPGA System: Harness, Builds, Flow -- Version 1.0

September 2026

Table of Contents

1 Document Information....8
1.1 Revision History....8
1.2 How to read this book....9
2 The System....10
2.1 What it is....10
2.1.1 Figure 1.1: The bench....10
2.2 The board....10
2.3 The two USB chips....11
2.4 The stack, top to bottom....12
2.5 What the host sees....12
3 The Harness....14
3.0.1 Figure 2.1: The harness block map....14
3.1 The host front-end and the bridge....14
3.2 The harness CSR....15
3.3 Stimulus and the memory model....16
3.4 The three instruments....17
4 One RTL, Three Builds....19
4.1 The rule....19
4.1.1 Figure 3.1: One source tree, three bitstreams....20
4.2 Why three, and why not two....20
4.3 What the builds measured....21
4.4 The knobs behind the rows....23

5 Host Tools and Flow.....25
5.1 The shared flow.....25
5.2 The host layers.....25
5.2.1 Figure 4.1: What each build measures, end to end.....27
5.3 One program, two targets.....27
5.4 Where the results go.....27

List of Figures

Figure 1.1: The bench.....10
Figure 2.1: The harness block map.....14
Figure 3.1: One source tree, three bitstreams.....20
Figure 4.1: What each build measures, end to end.....27

List of Tables

Table 1: Table 0.1: Document information....8
Table 2: Table 0.2: Revision history....8
Table 3: Table 1.1: The part and what the builds use of it....11
Table 4: Table 1.2: The stack....12
Table 5: Table 2.1: The bridge address map (monitor build)....14
Table 6: Table 2.2: Harness CSR, the registers that define a run....16
Table 7: Table 2.3: Stimulus and memory model....16
Table 8: Table 3.1: The three builds....19
Table 9: Table 3.2: Sign-off builds....21
Table 10: Table 3.3: The knobs....23
Table 11: Table 4.1: Flow targets (from make/fpga_flow.mk)....25
Table 12: Table 4.2: Per-build host programs....26
Table 13: Table 4.3: Reports....27

List of Waveforms

No waveforms in this document.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

1 Document Information

Table 0.1: Document information

Field	Value
Title	STREAM on the Genesys 2 – The FPGA System
Version	1.0
Date	2026-09-29
Scope	The STREAM characterization and monitor-validation system: board, harness, the three builds, host flow
Not in scope	The STREAM core itself (HAS/MAS under `projects/components/dma-ip/stream/docs/`); the measured results (`reports/`, `stable/MANIFEST.md`)
Status	Current; written against the tree at the stream BUG-018 close (commit 022fa0e3e)

1.1 Revision History

Table 0.2: Revision history

Version	Date	Change
1.0	2026-09-29	First edition, after the component-level consolidation (one harness, one top, three parameter sets) and the mon and obs sign-off builds of September 2026.

1.2 How to read this book

Chapter 1 is the bench. Chapter 2 is the bitstream around the DUT, block by block, with the bridge address map every host tool depends on. Chapter 3 is why there are three bitstreams from one

source and what each is for; it is the chapter to read before trusting a zero. Chapter 4 is the shared flow, the per-build host layers, and the one-program-two-targets rule. Every diagram is a Graphviz source under `assets/graphviz/`, rendered to PNG by `regenerate_all_graphviz.sh`.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

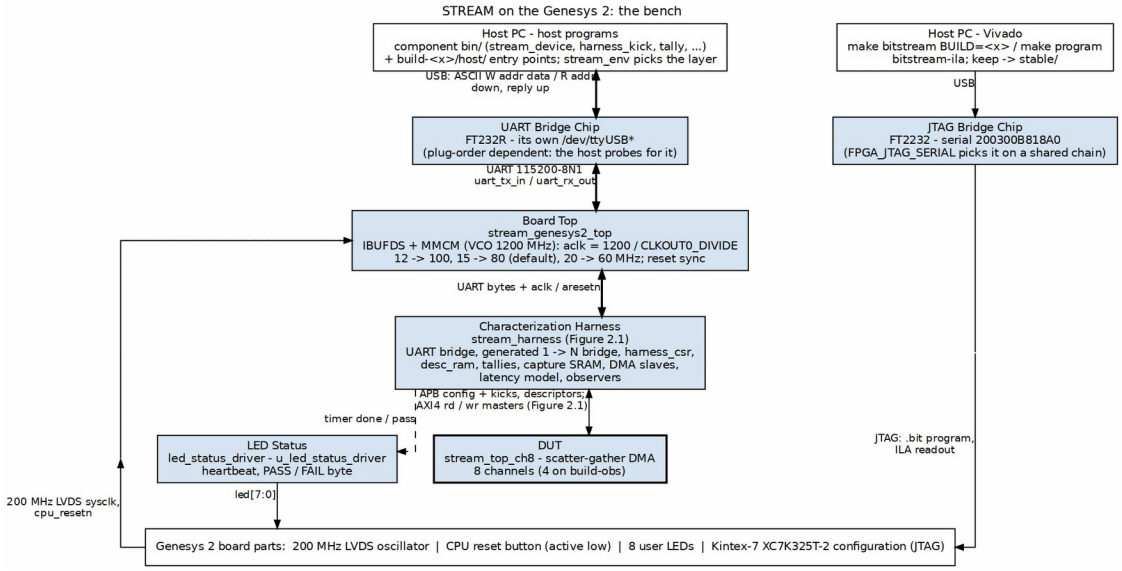
2 The System

2.1 What it is

The STREAM system on the Genesys 2 is two things sharing one bitstream source. It is a characterization harness: a synthesizable wrapper that feeds the STREAM scatter-gather DMA from an on-chip LFSR memory model, sinks its writes into a CRC checker, adds programmable memory latency, and counts every cycle of every interface. And it is a monitor-validation environment: the same wrapper carries the tally memories, capture SRAM and drains that let the host see every MonBus packet type from every agent in the DUT, whether the packets come from the in-core monitors or from interface observers standing outside the DUT.

Which of those it is on a given day is a question of which build is programmed. Chapter 3 is about that. The rest of this chapter is what does not change between them.

2.1.1 Figure 1.1: The bench



The bench: host, two USB chips, the FPGA

Source: 01_board_and_host.dot

2.2 The board

Digilent Genesys 2, Kintex-7 XC7K325T-2. The Nexys A7 version of this system (`stream_char_top`) was the original; the Genesys 2 top, `stream_genesys2_top`, adds an IBUFDS and an MMCM for the board's 200 MHz LVDS system clock and drops the seven-segment display for the eight user LEDs. Everything below the top is the same RTL on both boards.

The harness clock is a parameter, not a constant. The MMCM runs its VCO at 1200 MHz and the harness clock is 1200 divided by `CLKOUT0_DIVIDE`: 12 gives 100 MHz, 15 gives 80, 20 gives 60. `FPGA_CLK_HZ`, which sizes the UART divisor, the heartbeat and the timer, is derived from the same parameter, so the two can never disagree. The mon sign-off build ran at 60 MHz, which is where the full-cone monitor build closed with margin; the perf build is the one that cares about the clock, and it is small enough to run faster.

Table 1.1: The part and what the builds use of it

Resource	XC7K325T-2	The builds
Slice LUTs	203,800	mon sign-off 139,293 (68%); the two instruments together would need 217,761, which is why they are never built together (2026-10-03 re-measurement: 86,865 LUTs / 42.6%, WNS +1.914 ns; RTL has moved since the original sign-off)
Block RAM tiles	445	20 to 24 on the current reports
User LEDs	8	status bank; PASS and FAIL are one-byte patterns
Reset	red CPU reset button, active low	synchronized in the top

2.3 The two USB chips

The Genesys 2 carries JTAG on an FT2232 (serial `200300B818A0`, the one `FPGA_JTAG_SERIAL` names, because a Nexys A7 shares the chain in this lab) and the UART on a separate FT232R that enumerates as its own `/dev/ttyUSB*`. The host tools probe for the harness rather than pin a device node. The link is 115200-8N1 with the ASCII `W addr data` / `R addr` protocol, one 32-bit AXI4-Lite access per line.

2.4 The stack, top to bottom

Table 1.2: The stack

Layer	Module	Role
Board top	`stream_genesys2_top`	LVDS clock in, MMCM, reset sync, LEDs, the geometry generics
Harness	`stream_harness`	UART to AXI4-Lite bridge, the generated one-to-many bridge, harness CSRs, descriptor RAM, tallies, capture SRAM, DMA slaves, latency model, observers
DUT	`stream_top_ch8`	the STREAM DMA: APB configuration and kicks, eight channels (four on the obs build), one descriptor AXI4 master, read and write data masters, MonBus egress

The harness is component-level: one `stream_harness.sv`, one `stream_genesys2_top.sv`, one `harness_csr.sv`, one `axi_response_delay.sv`, one set of generated bridges, one Tcl set and one XDC, shared by every build. Every geometry default the harness uses comes from `stream_cfg_pkg`, which is the single source the board top and the simulation share. A literal in the harness is how the simulation and the board once drifted apart, with the perf characterization run at sixteen outstanding transactions against a board built at two.

2.5 What the host sees

One AXI4-Lite master, fanned out by a generated bridge to every slave in the harness: the STREAM APB window, the harness CSRs, the descriptor RAM, the MonBus error drains, the two tally memories and their configuration, the slave-monitor and observer register blocks, and the

capture SRAM. Chapter 2 has the map. Every register the host touches is reached by name through a register map generated from `regs/harness_csr.rdl`; a hand-kept Python table used to exist, drifted, and declared ten registers the RTL did not decode, which three host tools then read as zeros off a running board.

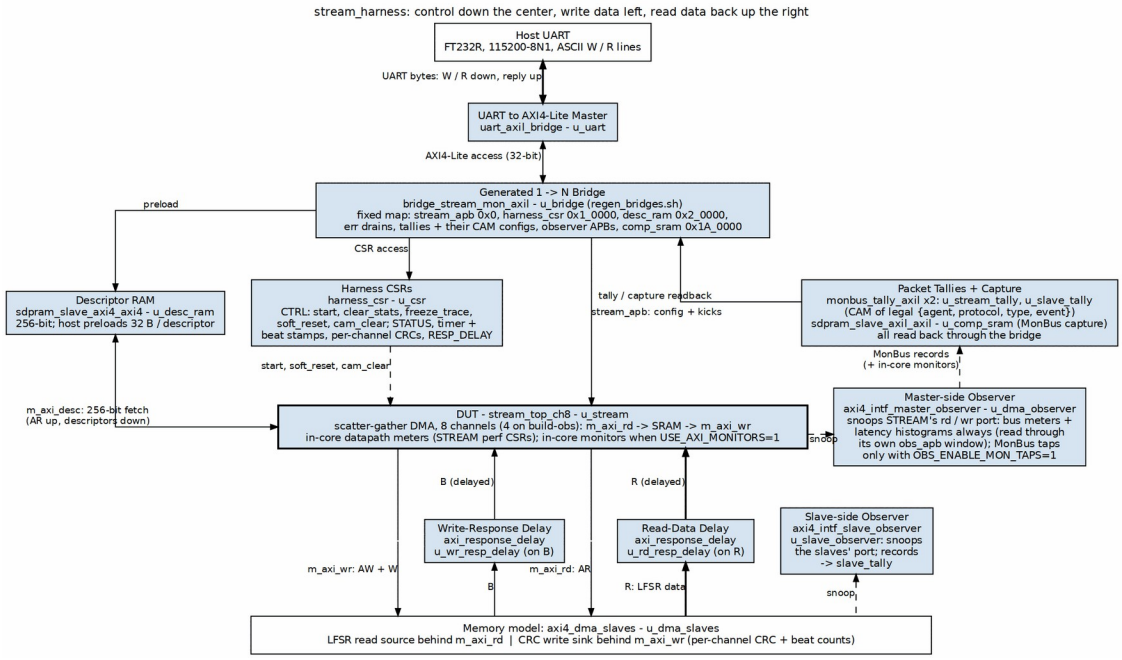
RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

3 The Harness

`stream_harness` is the bitstream minus the pins. Unlike the RAPIDS harness, whose host front-end was moved inside it late, this one was built with the UART bridge and the CSRs inside from the start, so the cosim and the board have always run the same host programs over the same bytestream. Figure 2.1 is the block map.

3.0.1 Figure 2.1: The harness block map



The harness: front-end, stimulus, DUT, the three instruments

Source: 02_harness_blocks.dot

3.1 The host front-end and the bridge

`uart_axil_bridge` turns the UART bytestream into one 32-bit AXI4-Lite master. A generated bridge (`bridge_stream_mon_axil`, from the bridge generator in `projects/components/fabric-gen-ip/bridge`) fans it out to every slave, converting protocol and width per port; it is regenerated by `bin/regen_bridges.sh` as the flow's PREBUILD step, so a slave added to the map is a generator input, not hand-written glue.

Table 2.1: The bridge address map (monitor build)

Base	Size	Slave
`0x0000_0000`	8 KB	`stream_apb`: the STREAM configuration window

Base	Size	Slave
0x0001_0000	4 KB	harness_csr
0x0002_0000	64 KB	desc_ram: descriptor preload, 32 bytes per descriptor
0x0003_0000	4 KB	stream_err: the MonBus IRQ FIFO drain
0x0004_0000	256 KB	stream_tally: the in-core monitors' packet tally
0x0008_0000	4 KB	dma_axil: tied off, reserved
0x0009_0000	4 KB	slave_err: the slave-monitor error drain
0x000C_0000	256 KB	slave_tally: the observers' packet tally
0x0010_0000	256 KB	stream_tally_cfg: tally CAM configuration and readback
0x0014_0000	256 KB	slave_tally_cfg
0x0018_0000	4 KB	slvmon_apb: slave-monitor configuration
0x0019_0000	4 KB	obs_apb: observer configuration
0x001A_0000	64 KB	comp_sram: MonBus capture memory, host download

Every MonBus-carrying AXI path is 64 bits wide. The descriptor read path is a direct 256-bit AXI4 from the DUT to desc_ram, bypassing the bridge, and the data masters are 128-bit into the pattern generator and CRC checker.

3.2 The harness CSR

harness_csr is the host's control and status surface, separate from the STREAM registers in the APB window. Its map is generated from regs/harness_csr.rdl; the RTL is hand-written against it and every external AXI4-Lite channel sits behind a skid buffer for timing.

Table 2.2: Harness CSR, the registers that define a run

Register	Role
CTRL	start (opens the measurement window), clear_stats, freeze_trace (stops capture writes), soft_reset, cam_clear (clears every CAM in the DUT's monitors and the tally)
STATUS	latched stream_irq, sticky any_error, sticky trace_overflow, clear_busy (poll to zero after clear_stats before the next capture)
DBG_WR_PTR, DBG_OVERFLOW	how much of the capture memory is written, and whether it overflowed
CRC_RD_EXPECTED, CRC_WR_EXPECTED, CRC_WR_COMPUTED	the source's pseudo-random CRC, the expected sink CRC, and what the sink actually wrote
meter and count registers	the per-interface cycle classification the perf host reads

soft_reset clears every instrument in the harness, which is what lets one programming run a long sequence of host programs without inheriting state; the runner hooks on the host issue it between programs.

3.3 Stimulus and the memory model

Table 2.3: Stimulus and memory model

Block	Role
axi4_dma_slaves	an LFSR read-source pattern generator behind m_axi_rd and a

Block	Role
	write-sink CRC checker behind `m_axi_wr`; both accept the next transaction on the last beat of the current one (an earlier two-state version inserted a dead cycle per burst and cost the datapath a phantom 6 percent)
`axi_response_delay` x2	a programmable memory-latency model on the read and on the write path, the knob of the latency sweeps
`desc_ram`	the descriptor store the host preloads and the DUT fetches from directly

3.4 The three instruments

The harness carries three ways of measuring, and Chapter 3 is about which build has which.

- **Bus meters**, always built. Per interface, every cycle of the window is productive, backpressure, starvation or idle. They sit outside the monitor gate on purpose: the perf build has no monitor and must still count. That separation regressed once, when the gate over-reached and the perf build reported zeros, and the monitor test now asserts the meters are non-zero in that flavour.
- **Tallies**, `monbus_tally_axil` twice. A direct count over every possible MonBus identity would be a 20,480-cell matrix of which about 245 tuples are legal; crossed with a 16-bit agent id it would be 99 percent empty. The tally instead loads the legal `{agent, protocol, packet type, event}` tuples into a CAM (`MON_N_PROFILE` slots, 32 on the sign-off builds) and counts densely, so the host can prove every agent emitted every packet type it may legally emit. `stream_tally` counts the in-core monitors, `slave_tally` the observers.
- **Capture**, `comp_sram`. The MonBus stream itself, compressed on the mon build, downloadable by the host and decoded against the cosim reference; `stream_err` and `slave_err` drain the error and IRQ FIFOs.

The two optional sources of packets are the DUT's in-core monitors (`USE_AXI_MONITORS`) and the interface observers standing on the DUT's masters and on the slaves (`OBS_ENABLE_MON_TAPS`). Both feed the same tally and capture machinery; they are never built together, and the reason is in Chapter 3.

---

4 One RTL, Three Builds

4.1 The rule

There is one harness, one board top, one CSR block, one latency model, one set of generated bridges, one Tcl set and one XDC, all at component level. `build-mon/`, `build-obs/` and `build-perf/` each hold a Makefile of variables and nothing else; the flow logic is the shared `make/fpga_flow.mk`. Which build you want is a question about what is under test, and the answer is a row in this table:

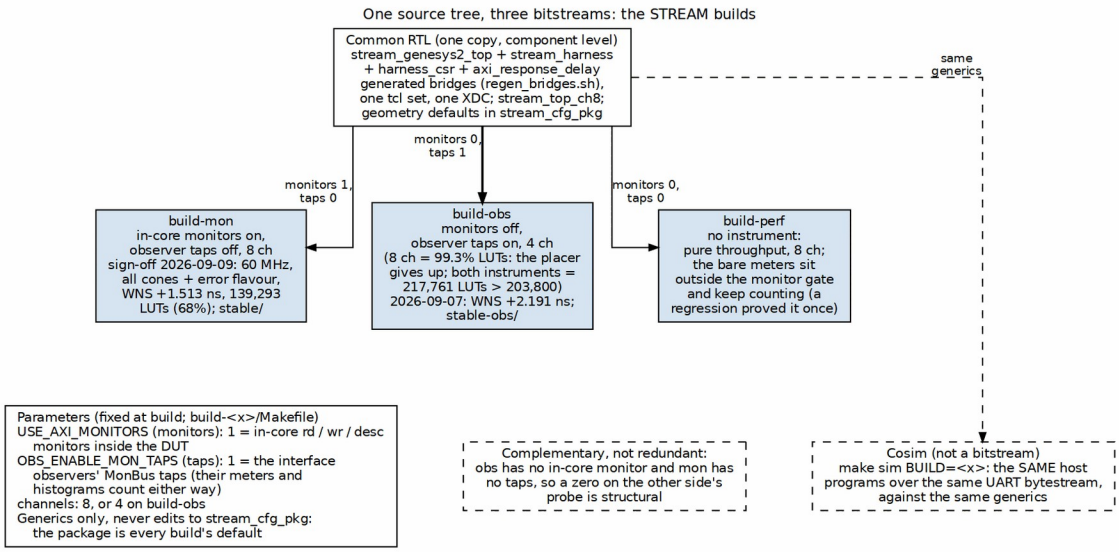
Table 3.1: The three builds

Build	USE_AXI_MONITORS	OBS_ENABLE_MON_TAPS	Channels	Measures
`mon`	1	0	8	the in-core monitors: coverage, compression, error and fault probes
`obs`	0	1	4	the interface observers, watching the same DUT from outside it
`perf`	0	0	8	the datapath itself, with no instrument in the way

Two near-identical harnesses, one per flavour, used to exist. They were retired because a variant that is a copy drifts: the board and the simulation once disagreed on the outstanding-transaction

limit for exactly that reason. Now a variant is a parameter, the parameter is a generic handed to Vivado, and `stream_cfg_pkg` stays every build's default; editing the package to retarget one build silently retargets the other two.

4.1.1 Figure 3.1: One source tree, three bitstreams



One RTL, the knobs, the three builds

Source: 03_build_variants.dot

4.2 Why three, and why not two

The mon and obs builds are complementary, not redundant. They exercise the monitor code from two directions: the in-core monitors see transactions from inside the DUT, the observers see them on the interfaces outside it, and both push their packets through the same tally and capture machinery. So build-obs has no in-core STREAM monitor at all, and a zero on any `mon_*` probe there is structural; build-mon has its observer taps off, and a zero on an observer probe there is equally structural. Reading either zero as a defect has cost an afternoon before, which is why the Makefile says so in capitals.

They cannot be one build. With every monitor cone and both observers at eight channels the design needs 217,761 LUTs against the 203,800 on the part, and the placer never runs. Even the obs build alone does not fit at eight channels: it reached 99.33 percent of the LUTs and "could not place all instances", so obs is pinned at four (six does not route either). The eight-channel design point is perf's requirement; the obs channel count is incidental, not a traded-away requirement.

The perf build exists so that a number about the datapath is a number about the datapath. Its bus meters are instantiated outside the monitor gate and keep counting with every instrument removed; that arrangement is what lets the design fit a small part, and it is guarded by a test after it regressed once.

4.3 What the builds measured

Table 3.2: Sign-off builds

Build	Sign-off	Clock	Result
`mon`	2026-09-09, `stable/MANIFEST.md`	60 MHz	all cones plus the error flavour, compression built, 8 channels, `MON_N_PROFILE=32` : WNS +1.513 ns, 139,293 LUTs (68.35%), 0 failed nets; 512 packets in 516 slots on the board, 66.4% smaller than the raw encoding, CRC match, cosim reference within 4 points (2026-10-03 re-measurement: 66.0% / 1.02 slots/packet, within data-dependent noise; bit sha256 `aceec688043ad630879779ddae211814fcc4dc457365d7c06ec95b8a4f348e6b` , WNS +1.914 ns, 86,865 LUTs)
`obs`	2026-09-07, `stable-obs/MANIFEST.md`	as built	4 channels, 32-entry tally CAM, WNS +2.191 ns, archived the day a `make clean-all` destroyed the

Build	Sign-off	Clock	Result
			previous obs bitstream and it turned out obs had been synthesizing at 100.01 percent of the LUTs and placing by luck (2026-10-03 re-measurement: `build-obs/results/obs_board_2026-10-03.txt`, bit sha256 `60696ace63fed2c77ad4d3f1f511d06dd85801c9b5a9dac1d4078454192bfa5f`, WNS +3.925 ns, 73,809 LUTs; the observer instrument changed 2026-09-27, commit 78cddb5e2 — lite taps, no perf/debug cone, caps0 bit4/bit5 read 0)
`perf`	current reports on disk	as built	68,139 LUTs (33.4%), 24 BRAM tiles in `build-perf/fpga/reports`; the sweeps behind `reports/perf` and `reports/ext_addr`

Build	Sign-off	Clock	Result
			essing (2026-10-03 re-measurement: build-perf/results/perf_sweep_2026-10-03.{csv,json}, bit sha256 d26f9b7b3ac534a1507ab5a519e728d541b1045cf67000329fbedf77e3b9512c, WNS +1.175 ns, 64,305 LUTs)

The stable/ directories hold exactly one build each, with the bitstream, the Vivado reports for that bitstream, the board results measured on it and a manifest naming the tree SHA it was built from. They exist because make clean-all in a build directory deletes tracked bitstreams and reports, and that has destroyed a verified bitstream more than once; stable/ is a sibling of the build directories, outside that blast radius.

4.4 The knobs behind the rows

Table 3.3: The knobs

Knob	Where	What it selects
STREAM_NUM_CHANNELS	build Makefile	channel count: 8 for mon and perf, 4 for obs
USE_AXI_MONITORS	build Makefile, DUT generic	the in-core rd/wr/descriptor monitors, their CAMs, and the compression pipeline (gated on this flavour only)
OBS_ENABLE_MON_TAPS	build Makefile, harness generic	arm the observers' MonBus event taps; the observers' meters and histograms count without them
MON_ERROR_FLAVOR, MON_CONES	build Makefile	which monitor cones are built; the sign-off is the union including error
MON_N_PROFILE	build Makefile	tally CAM depth
CLKOUT0_DIVI	board top	100, 80 or 60 MHz harness clock

Knob	Where	What it selects
`DE`	parameter	
`stream_env`	host	which build's `host/` layer goes on the Python path (default `mon`; perf sets it explicitly so a perf program cannot import mon's copy of a sibling)

5 Host Tools and Flow

5.1 The shared flow

The area Makefile is a dispatcher: `make bitstream BUILD=obs` runs the target in `build-obs/`, whose Makefile is variables plus an include of `make/fpga_flow.mk`. The flow's targets are therefore identical across the three builds and across the other Genesys 2 and Nexys A7 systems that use it.

Table 4.1: Flow targets (from `make/fpga_flow.mk`)

Target	What it does
`lint`, `flat-filelist`, `lint-decl-order`	Verilator lint of the whole harness against the expanded filelist, before any Vivado run
`prebuild`	the build's PREBUILD step: here, regenerate the bridges
`project`, `synth`, `bitstream`	create the Vivado project; synthesis only with utilization and failing-path reports; the full flow with every report
`bitstream-ila`	the same design plus an ILA on the marked debug nets
`sim`	this build's harness cosim (cocotb, Verilator)
`program`	flash the board, naming the file, falling back to the hold copy after a clean
`run SEQUENCES="..."`, `seq-list`	drive the programmed board through the area's sequences
`utilization`, `timing`	print the latest reports
`keep`	copy the bitstream to `RDS_HOLD_DIR` outside the repo and the reports to `../stable/`
`clean`, `clean-build`, `clean-all`	remove Vivado artifacts, cosim trees, everything; `clean-all` is what `stable/` protects against

5.2 The host layers

Two levels of Python. The component `bin/` holds what every build uses: `stream_device.py` (the UART link and register access by name), `harness_kick.py` (staging the channel kicks), `bus_meters.py` (the meter packing, one definition shared with the cosim so both measure the same thing), `tally.py` and `dump_monbus.py` (reading the tallies and decoding a capture),

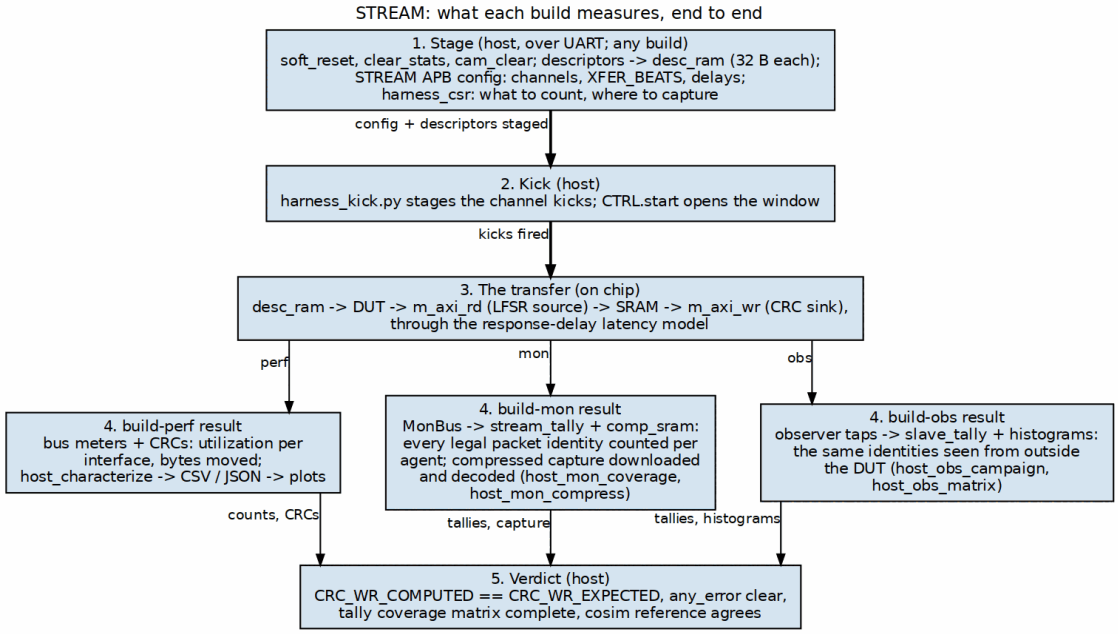
characterization.py and stream_ext_suite.py (the campaign runners), stream_monitors.py (monitor and observer configuration), and the plotting scripts. Each build then has its own host/ with the programs that only make sense on that bitstream:

Table 4.2: Per-build host programs

Build	Programs
perf	host_characterize, host_bus_meters, host_desc_perf, host_rw_perf, host_ext_char, host_ext_soak, host_probe_multichannel, host_verify_descriptors, host_status, host_perf_json_to_csv
mon	host_mon_coverage, host_mon_compress, host_mon_matrix, host_mon_err_probe, host_mon_fault_probe, host_reg_walk
obs	host_obs_campaign, host_obs_matrix, host_reg_walk

stream_env selects which build's host/ goes on the Python path. Its default is mon; build-perf sets it explicitly because without that a perf program importing a sibling from its own directory would get mon's copy, and it would appear to work, since Python also puts the running script's own directory first.

5.2.1 Figure 4.1: What each build measures, end to end



The three measurement paths

Source: 04_measurement_path.dot

5.3 One program, two targets

The same host program runs against the cosim and against the board. The cocotb harness testbench presents the UART bytestream to `stream_harness` exactly as the FT232R does, the harness carries the bridge and the CSRs, and the generics are the build's. A monitor coverage result on the board is therefore compared with the same program's result in cosim, and the sign-off manifest records both (512 packets in 516 slots on the board against 1.12 slots per packet in cosim, for the compression program). A host program that only works on one target is a defect in the harness or the program, not a difference in kind.

The sign-off discipline before a build is the same on every flavour: the FULL-level sim suites of all three builds green on a verified-clean tree, recorded in the manifest with the tree SHA, so the bitstream is reproducible from the recorded source.

5.4 Where the results go

Table 4.3: Reports

Report	Source build	Content
`reports/perf`	perf	utilization sweeps: channels, descriptors,

Report	Source build	Content
		sizes, latency (the CSV data under `docs/data/`)
`reports/compression`	mon	MonBus compression on the board against the cosim
`reports/area`	any	what the monitors and the readback logic cost in LUTs per channel count
`reports/ext_addressing`	perf	the extended row/column addressing campaign
`stable/results`	mon	the board sweeps measured on the sign-off bitstream

Each report directory has its own README and, where a deliverable exists, the versioned DOCX and PDF built through the house pipeline.